SERVICE CATALOG

New Programme Skill

Service Catalog — Technical Design Walkthrough

Design-first spec · pre-requirements

D55 Consulting

July 2026

Background

D55 wants a repeatable way to add a new service Programme to its Service Catalog. Every programme is built around a free assessment that determines a prospective client's current capability, then recommends the paid modules that close their gaps — Assess, Teach, Prove, Scale.

The AI-DLC programme was built by hand and became the reference for the pattern. Reproducing that by hand for each new programme is slow and inconsistent. The New Programme skill automates it: from a rough idea (and optionally a client's scores) it produces the whole asset set — assessment dimensions, a module library, per-module HTML and PDF assets, an internal runbook spreadsheet, an assessment questionnaire spreadsheet, an interactive questionnaire that renders a radar chart and recommends modules, and an elevator-pitch deck.

The defining feature is a six-persona sub-agent critique loop that refines each major artefact against internal (D55) and external (client) viewpoints before it reaches a human gate — so the user reviews a strong draft, not a first pass.

> ### What changes, in one line
>
> A hand-built, one-off programme becomes a manifest-driven skill that scaffolds, authors, critiques, and renders a complete programme — reusing the existing render engine and deliverables skills rather than reinventing them.

How it works

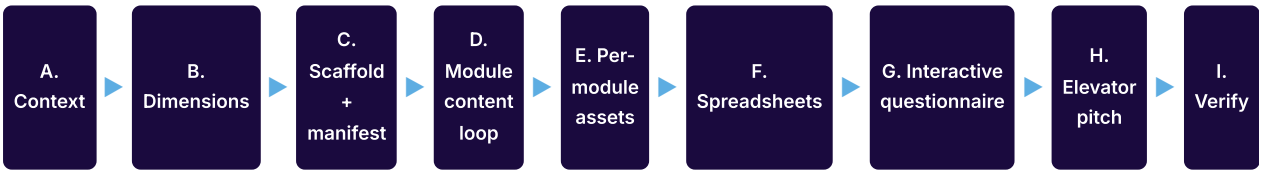
Three layers. An orchestrator (the new-programme skill) owns the phase flow and the human gates. Producer sub-skills each build one artefact type, mostly reusing what already exists (programme_engine.py, deliverables-toolkit, summary-presentation). A critique panel of six sub-agents refines artefacts between production and each human gate.

Everything is driven from a single machine-readable manifest (programme.yaml) generated at scaffold time. Docs, spreadsheets, and the interactive questionnaire all render from it, so the tooling never drifts from the content.

The skill runs in one of two modes: template mode maintains the canonical, reusable catalog entry; client-instance mode clones the template and scopes it to one client's assessment scores.

The programme build flow

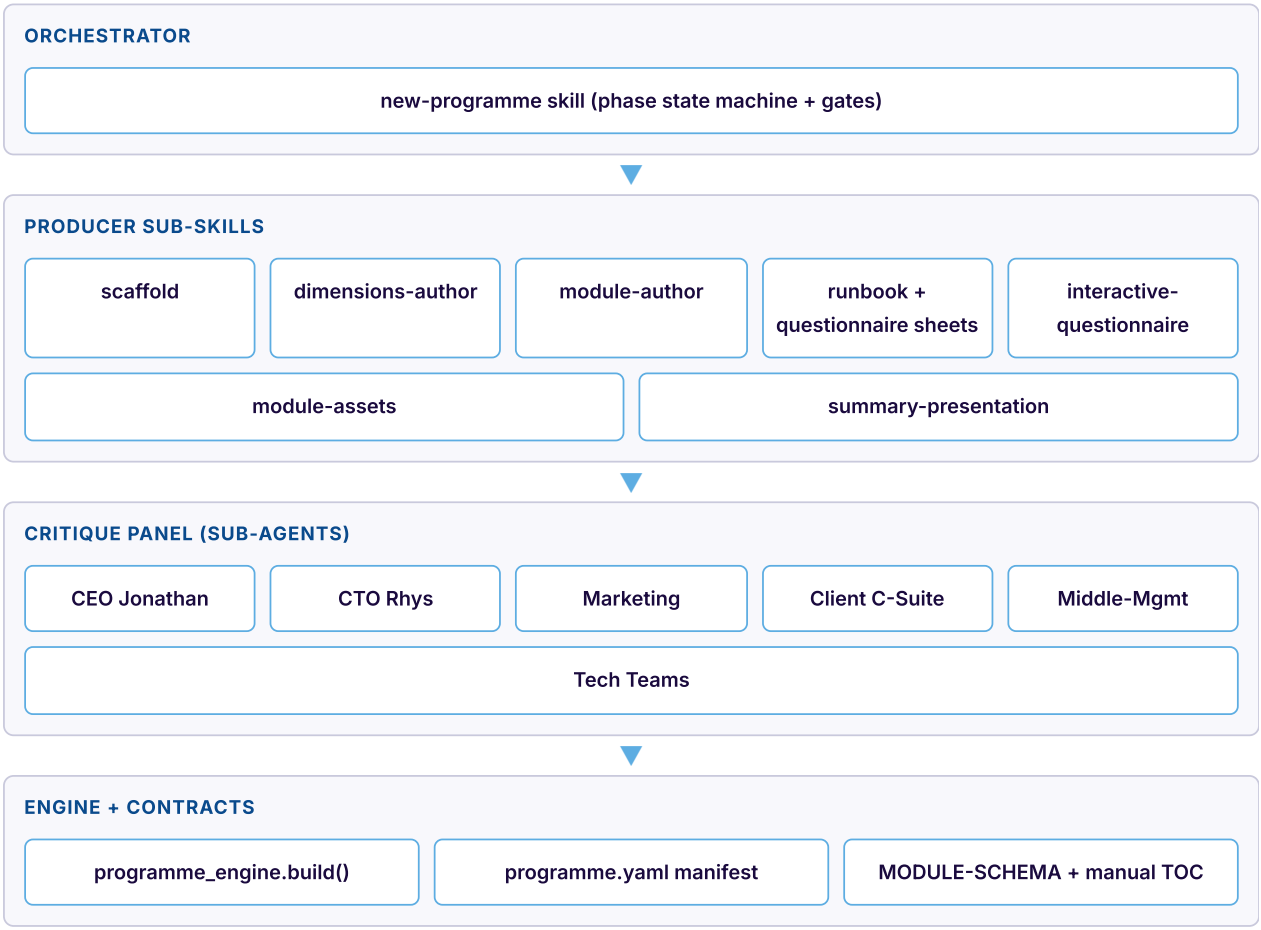
Nine phases from idea to a verified asset set. A six-persona critique loop runs before the human gate at the context, dimensions, module, and pitch phases (not shown as steps here — see the critique loop section).



Phases A, B, D and H each pass through the critique loop, then a human 'Happy?' gate, before advancing.

Architecture

The orchestrator delegates rendering to producer sub-skills and refinement to the critique panel. A shared programme workspace (files on disk) is the single source of truth every sub-agent reads from and writes to.



Producer sub-skills reuse the config-driven render engine; contracts keep the layers consistent.

The six critique personas

Each persona is a sub-agent with its own lens and scorecard. Internal personas hold a higher bar (>= 4/5); external personas check credibility (>= 3/5).

PERSONA	LENS	CARES MOST ABOUT
Jonathan — D55 CEO	Internal / commercial	Strategic fit, brand, margin, is this a sellable programme
Rhys — D55 CTO	Internal / delivery	Technical credibility, can a consultant run this, delivery risk
Marketing	Internal / GTM	Elevator pitch, funnel, the free-assessment hook, differentiation
Client C-Suite	External / buyer	ROI, risk, why you / why now, board-defensibility
Client Middle-Management	External / feasibility	Disruption to my team, workload, what this means on Monday
Client Technical Teams	External / credibility	Is this real or vendor fluff, depth, respect for how engineers work

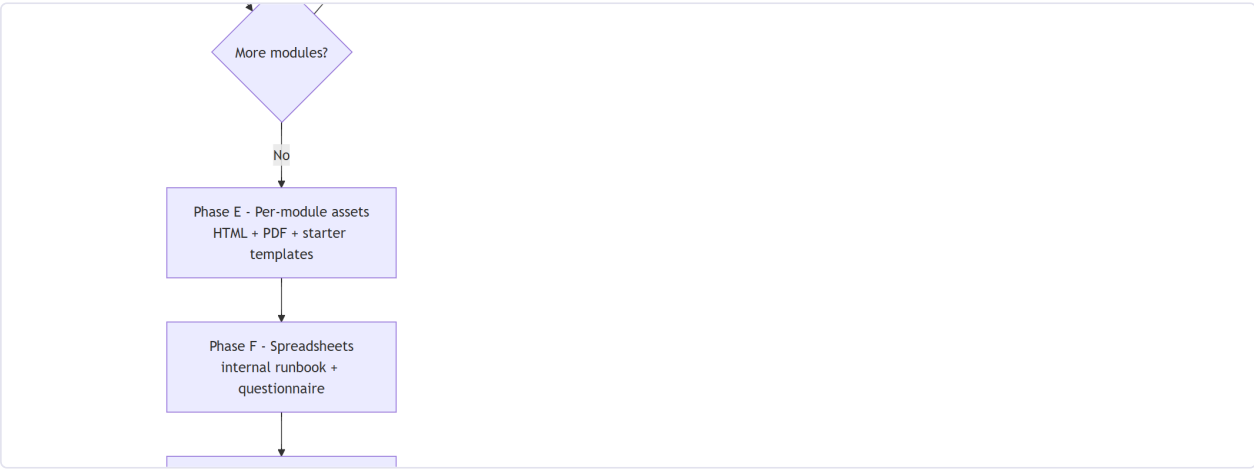
The critique loop

Each critiqued phase runs an autonomous refine loop before the human gate. Relevant personas critique the draft in parallel; the aggregator dedupes and ranks their findings and splits them into addressable-now (fix in-file) versus parked (needs a person or decision — never counted against the score).

Three guards guarantee the loop terminates: a maximum iteration cap (default 3, matching the established process), pass gates (primary personas meet their threshold and there are zero open blockers), and stall detection (if the backlog stops shrinking, the loop escalates rather than spinning). On pass or escalation the refined artefact — plus a critique summary — goes to the human 'Happy?' gate.

- PASS — thresholds met, no blockers → hand to the human gate
- ITERATE — apply the top-ranked addressable findings, then re-critique
- ESCALATE — cap hit or backlog stalled → stop, surface open items to the human

The critique loop (rendered)



Relevant personas critique in parallel; the aggregator triages addressable vs parked; gates and an iteration cap guarantee the loop terminates before the human review.

Which personas critique which phase

Primary personas (score gates) vs contributing vs light-touch, per artefact.

ARTEFACT (PHASE)	PRIMARY (GATES)	CONTRIBUTING
A. Context / positioning	CEO, Marketing, C-Suite	CTO
B. Dimensions / questions	CTO, Tech Teams, Middle-Mgmt	C-Suite
D. Module content	CTO, Middle-Mgmt, Tech Teams	CEO, Marketing, C-Suite
G. Interactive questionnaire	Marketing, C-Suite	CEO, Middle-Mgmt, Tech
H. Elevator pitch	CEO, Marketing, C-Suite	—

Self-sufficiency & portability

A hard constraint: the unit of portability is the skill directory. Any skill in this feature must run after being zipped and dropped into another repo — no reach-back into analysis/, no shared repo-root modules, no absolute paths. Portability wins over DRY: if two skills need the same engine, each vendors its own copy (or they ship together as one skill-set).

So a skill stops being a lone .kiro/skills/<name>.md file and becomes a bundle directory that carries everything it needs:

- engine/ — the vendored, self-contained render + spreadsheet + questionnaire engines
- templates/ — manifest, module, dimensions, and manual-TOC skeletons
- personas/ — the six critique rubrics
- assets/brand/ — default D55 logo + background (overridable per programme)
- examples/ai-dlc/ — a trimmed worked example (patterns, not full prose)
- requirements.txt — bundle-local Python deps (openpyxl, playwright, pypdf, hypothesis)

> ### How we prove it
>
> A portability check (Property 13) copies a bundle to a temp dir outside the repo, runs it on its bundled example, and asserts nothing resolves into analysis/, the repo root, or an absolute path. Paths resolve relative to the bundle; the output location is a parameter.

Key design decisions

A handful of choices shape the rest of the build.

DECISION	CHOICE	WHY
Source of truth	Manifest-first (programme.yaml)	Docs and tools render from one machine-readable spine; keeps the interactive tool in sync
Quality mechanism	Six-persona critique loop before the human gate	The user reviews a refined draft; captures external client viewpoints the offering is sold into
External thresholds	Lower than internal (3 vs 4)	External audiences stress credibility; holding them to 5/5 would loop forever on pilot-only fixes
Build vs reuse	Thin producer sub-skills over vendored engine	programme_engine, deliverables-toolkit and summary-presentation patterns do the heavy rendering
Modes	Template library + client-instance clone	Maintain one canonical programme; scope per client by scores without editing the template
Portability over DRY	Self-sufficient skill bundles	Vendor engine/schema/assets/example so a skill zips and runs elsewhere with no external-folder deps

The manifest, at a glance

programme.yaml ties every artefact together and is validated on write.

SECTION	HOLDS	NOTES
programme	Slug, name, one-liner, phases, commercial model	The programme's identity and pitch
brand	Palette, logo, background	Feeds BrandConfig for the render engine
dimensions	The radar axes (name + short + rubric ref)	Join key: names must match module dimensions_covered
modules	id, title, dimensions_covered, manual_section, trigger	Mirrors module.md frontmatter for fast consumption

Join-key contracts

The programme is only tooling-consumable if three joins hold; a validator hard-stops the build on any violation.

JOIN	AUTHORITY	MUST MATCH
Dimension coverage	dimensions[].name	dimensions_covered[] in each module.md
Manual mapping	manual TOC section titles	manual_section in each module.md
Scoring	dimensions[].name	DimensionScore.dimension (bijection — scored once)
Criticality	a module's dimensions_covered	trigger.critical_dimensions[]

Testing note

The design defines 13 correctness properties — manifest integrity, scoring bijection, recommendation monotonicity, loop termination, gate integrity, self-containment, bundle portability, and more. These drive unit tests, property-based tests (hypothesis), and rendered-output verification (measure the DOM with Playwright; read PDFs back with pypdf). Loop termination and recommendation monotonicity are the highest-value properties to prove.

Open questions

Carried into requirements for confirmation.

QUESTION	WORKING ASSUMPTION
First-build scope	Ship critique-loop + manifest against AI-DLC first, then generalise the engine
Critique cost / latency	Cap iterations; make panel membership per-phase configurable
Runbook spreadsheet shape	Stages, activities, owners, inputs/outputs, decision points — confirm columns
Interactive questionnaire hosting	Self-contained HTML only for now; hosted/booking version later
External programme name	Skill prompts for it but allows deferral (AI-DLC's is still TBD)